Financial Pipeline Automation — System Architecture & Technical Trace

This document provides a comprehensive breakdown of your financial data processing and analysis pipeline. The system operates as a multi-stage data engine that cleans messy accounting spreadsheets, uses semantic filters and large language models to standardize financial labels, verifies mathematical integrity, and outputs structured analytical reports.

---

1. `llm_client.py` — The AI Communication Bridge

Purpose

This file functions as the dedicated API wrapper. It establishes a connection with an external large language model (LLM) server (such as Groq or OpenAI) and forces the model to respond strictly with structured JSON objects instead of free-flowing conversational text.

How It Works Step-by-Step

1. **Environment Setup**: The script pulls the required configuration options directly from your system environment variables (`LLM_MODEL`, `LLM_BASE_URL`, `LLM_API_KEY`).
2. **Client Initialization**: It creates an instance of the `OpenAI()` client pointing to the configured gateway.
3. **Structured Querying**: The `llm_json()` function takes a prompt and submits it to the model.
4. **Enforced JSON Output**: It passes the parameter `response_format={"type": "json_object"}`. This forces the LLM's raw token picker to only generate text that parses cleanly as a JSON object.
5. **Retry Logic Loop**: If the LLM produces a broken string, a `try-except` block catches the `JSONDecodeError`, discards the bad output, and retries the request up to two more times before failing.

Real Project Example

When `label_mapper.py` needs to find your revenue row, it passes a request to this file. `llm_client.py` speaks to the model and passes back a clean dictionary structure like this:

```
{
  "revenue": {
    "labels": ["Revenue from Contract with Customer, Excluding Assessed Tax"],
    "reason": "Matched because it represents total core sales contract inflows."
  }
}
```

---

2. `label_mapper.py` — The Intelligent Mapping Optimizer

Purpose

This file acts as a smart dictionary builder. Because different companies call the same financial line items by completely different names, this script coordinates an iterative process with the LLM to map raw CSV rows into standardized keys, while validating that the numbers make physical accounting sense.

How It Works Step-by-Step

1. **Cache Inspection**: Before doing any computational work or API calls, it checks if a local file like `mapping_gm.json` already exists. If found, it instantly returns the saved data to avoid duplicate API fees.
2. **Raw Extraction**: It uses `planning.load_statement()` to fetch the raw row names from your balance sheet and income statement CSVs.
3. **Gap Assessment**: It checks which standardized keys (like `short_debt` or `revenue`) are missing values for the most recent period under the standard alias configuration (`planning.ALIASES`).
4. **Iterative LLM Proposing**: For any missing items, it calls `propose()`, sending the full list of raw text labels to the LLM via `llm_json`.
5. **Multi-Pass Validation**: It feeds the proposed map into `validate_numbers()`. This helper function calculates total balances across your timelines and runs logical cross-checks:
    - It checks if Total Assets = Total Liabilities + Total Equity within a 3% tolerance.
    - It checks if Revenue - COGS = Gross Profit within a 2% tolerance.
    - It flags any year-over-year revenue jumps exceeding 90% or debt jumps exceeding 70% as high-probability mapping anomalies.
6. **Feedback Loop Resolution**: If a math rule fails, the script appends the exact error message to a `feedback` text string and re-runs the LLM prompt loop with instructions on what went wrong, giving the model up to 3 attempts to correct its mapping choices.
7. **Flagging Unreliable Metrics**: Any metrics affected by a failed validation test are added to an `unreliable` dictionary to alert downstream reports.

Real Project Example

If an automaker's balance sheet lists `Property, Plant, and Equipment and Finance Lease Right-of-Use Asset, after Accumulated Depreciation and Amortization`, the code flags this as missing from default configurations.

- **Pass 1:** The LLM maps it to the standard `ppe` key.
- **Pass 2:** `validate_numbers()` verifies that adding this number to current assets aligns with `Total Assets`.
- **Pass 3:** The mapping is finalized, saved to the cache file, and exported to your pipeline.

---

3. `planning.py` — The Analytical Processing Engine

Purpose

This file is the mathematical core of the pipeline. It reads raw CSV statements, handles fallback row-stitching, pivots layout structures so that dates become table indexes, runs financial safety ratios, tracks trend momentum trajectories, and writes the analytical final dashboard.

How It Works Step-by-Step

1. `load_statement`: Imports CSV data, automatically transposes the dataframe if dates are rows instead of columns, strips tag prefixes (like `us-gaap_`), and sorts dates chronologically with the newest period on the far left.
2. `pick`: Acts as a data-patching manager. If your primary line-item row contains blank `NaN` openings for certain years, it searches fallback alias rows to slice and stitch values into those specific missing historical dates.
3. `build_items`: Groups your parsed series arrays into a standardized DataFrame, automatically deriving total liabilities (Assets - Equity) or gross profit (Revenue - COGS) if they are missing.
4. `compute`: Applies safe division math via `sdiv()` (which safely converts zero-denominators to `NaN` to prevent system crashes) to calculate 16 distinct metrics covering liquidity, solvency, margins, and growth.
5. `classify`: Evaluates individual calculated points against your `BENCH` reference thresholds. It checks whether a higher value is better (`hib: True`) or worse (`hib: False`) to automatically classify each ratio as a `"strength"`, `"weakness"`, or `"neutral"`.
6. `trend`: Looks back across the 4 most recent periods. If all year-over-year deltas are moving in the same direction, it pairs that trajectory with your asset performance rules to assign operational tags like `"rising_improving"` or `"falling_deteriorating"`.
7. `analyze`: Orchestrates the entire script, builds structural meta-logs, handles edge cases (like marking metrics `"stale"` if their data lags behind the rest of the sheet), and writes the completed profile to `client_analysis.json`.

Real Project Example

When computing General Motors' profile:

- `compute()` runs your liquidity formula: Current Assets / Current Liabilities.
- `classify()` takes the resulting ratio of `1.15`, references your `BENCH["current_ratio"]` rules (Weak: `1.0`, Strong: `1.5`), and classifies it as `"neutral"`.
- `trend()` checks the last 4 years of data. If the ratio has dropped sequentially from `1.30` to `1.15`, it labels the trajectory `"falling_deteriorating"`.

---

4. `run_analysis.py` — The Top-Level Controller

Purpose

This script acts as the operational conductor for enterprise processing. It acts as an orchestrator that sets execution scopes for specific corporate clients, passes structural configurations, calls the mapping engine, and triggers the core analysis pipeline.

How It Works Step-by-Step

1. **Configuration Definition**: It sets the client targeting metadata variable (`client = "GM"`) and paths out localized spreadsheet file destinations.
2. **Context Note Injection**: It appends a static dictionary note warning that GM and Ford statements include massive consolidated automotive captive financing arms (`GM Financial` / `Ford Credit`), noting that their leverage matrices cannot be accurately cross-compared against pure asset manufacturers.

3. **Mapping Trigger**: It calls `label_mapper.resolve()`, passing your LLM execution parameters and naming a dedicated local caching file.
4. **Error Collection Construction**: It converts any LLM decisions and numeric validation failures into structured logging text lines.
5. **Execution Call**: It boots `planning.analyze()`, passing the newly resolved aliases and collected verification alerts, compiling your final analysis file: `gm_analysis.json`.

Real Project Example

Running `python run_analysis.py` executes the entire pipeline automatically:

```
[Start] -> Loads GM CSV paths -> resolves/caches labels via LLM -> verifies math
identity loops -> calculates margins & trends -> outputs 'gm_analysis.json' ->
[Done]
```

---

5. `financial_gate.py` — The Enterprise Risk & Governance Gate

Purpose

This script provides an interactive terminal-based data governance gate. Built as a command-line tool, it applies strict architectural constraints on financial data ingest, enforces data lineage, prevents row double-counting, and mandates explicit human approval before freezing enterprise datasets.

How It Works Step-by-Step

1. **Dynamic File Ingestion**: Parses execution parameters via `argparse`. It reads both financial CSV inputs and handles layout detection, identifying date string formatting (like `MM/DD/YYYY` or simple year digits) and resolving table layouts.
2. **Deterministic-First Mapping Layer**: Unlike the client pipeline, this script runs an optimization layer first. It checks for exact string matches against standard labels, applies anchored regular expressions (e.g., `r"^(total )?current assets$"`), and uses fuzzy similarity scoring (with a 0.86 threshold) before allowing optional LLM proposals.
3. **Strict Semantic Guardrails (`violates`)**: It applies strict compliance checks to prevent AI hallucination or user classification errors:
    - `short_debt` **Guard**: Blocks any row containing phrases like "*excluding current maturities*" or "*noncurrent*".
    - `total_liabilities` **Guard**: Instantly rejects rows containing "*liabilities and equity*" because they combine distinct asset classes.
    - **Expense Guard**: Blocks lines with words like "*cost of*" or "*expense*" from being mapped to top-line `revenue`.
4. **Data Lineage Enforcement**: Validates that no single CSV row is mapped to more than one target field, preventing double-counting.
5. **Comprehensive Integrity Checks**: Checks data consistency across periods and alerts users to anomalies like negative asset value lines or severe year-over-year balance jumps.
6. **Interactive Terminal Choice Logic**: If multiple rows contain conflicting values for the same accounting period, the script pauses execution and lists options inside the terminal interface, prompting the user to

manually select the correct entry.
7. **Audit-Trail Export**: Once authorized by a user typing `y`, the tool saves three distinct files inside an isolated, timestamped folder:
    - `gate_report.json`: The complete system audit profile.
    - `approved_financials.json`: The clean, calculated financial data array.
    - `mapping_approved.json`: The frozen mapping configuration used for the run.

Real Project Example

If you run a balance sheet that lists both `Short-term Borrowings` and `Current Portion of Long-term Debt` under a messy header structure:

1. The script flags a `needs_choice` status because multiple rows map to `short_debt`.
2. It lists them in the terminal window:

```
Field 'short_debt' (short-term borrowings): rows disagree.
  1. Short-term Borrowings  (latest 5000, 7 periods)
  2. Current Portion of Long-term Debt  (latest 2000, 7 periods)
Which row is correct? [1-2, Enter = 1]:
```

3. Once you make your selection, it runs the mathematical checks, confirms your sheets match, and saves the verified dataset to your output directory.

---

6. `dump_debt.py` — The Target Inspection Debugger

Purpose

This is a lightweight debugging utility script. It bypasses the analysis pipeline to scan a raw CSV balance sheet file and print timeline arrays for any rows related to debt, leases, or property infrastructure, giving you a quick way to inspect data values during testing.

How It Works Step-by-Step

1. **Target Word Definition**: Creates a tracking tuple filled with target words: `("debt", "borrow", "lease", "notes", "property", "equipment")`.
2. **Raw File Load**: Calls `planning.load_statement()` to clean and extract the index rows of your targeted corporate client spreadsheet file.
3. **Keyword Scan**: Loops through every row name inside the file's index, converting the text to lowercase to ensure matching consistency.
4. **Console Output**: If any target keyword is found in a row name, it prints the row label alongside its numerical value array, rounded to the nearest integer.

Real Project Example

Running this script against a raw GM balance sheet output instantly prints targeted structural lines straight to your terminal window for rapid verification:

```
LongTermDebtAndFinanceLeasesNoncurrent
PropertyPlantAndEquipmentNet
```